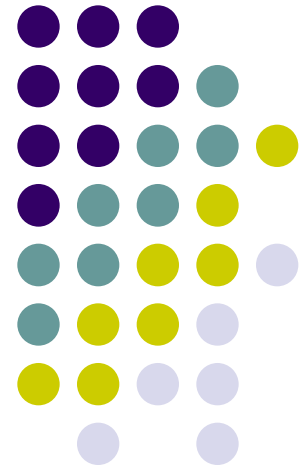
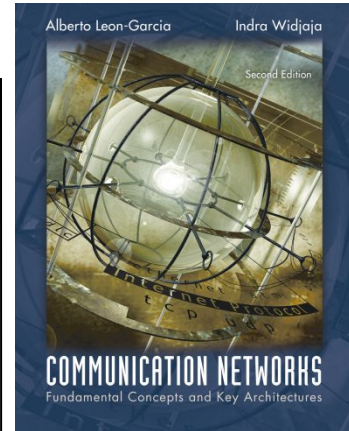
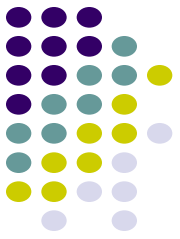


Applications and Layered Architectures

Protocols, Services & Layering

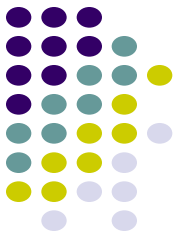


Learning Outcomes



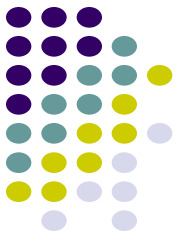
- At the end of this section, you should be able to
 - Explain the purpose of layering and protocols
 - State the layers in the TCP/IP protocol stack
 - Describe the role and services of DNS, HTTP, TCP and UDP protocols

What is an internet?



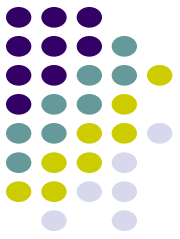
- A set of *interconnected networks*
- The Internet is the most famous example
- Networks can be completely different
 - Ethernet, ATM, modem, ...
 - (TCP/)IP is what links them

What is an internet? (cont)



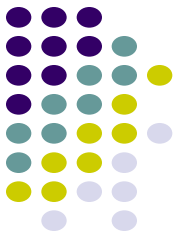
- *Routers* (nodes) are devices on multiple networks that pass traffic between them
- Individual networks pass traffic from one router or endpoint to another
- TCP/IP hides the details as much as possible

Layers, Services & Protocols



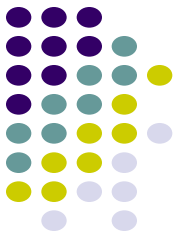
- The overall communications process between two or more machines connected across one or more networks is very complex
- **Layering** partitions related communications functions into groups that are manageable
- Each layer provides a **service** to the layer above
- Each layer operates according to a **protocol**
- Let's use examples to show what we mean

ISO/OSI Network Model



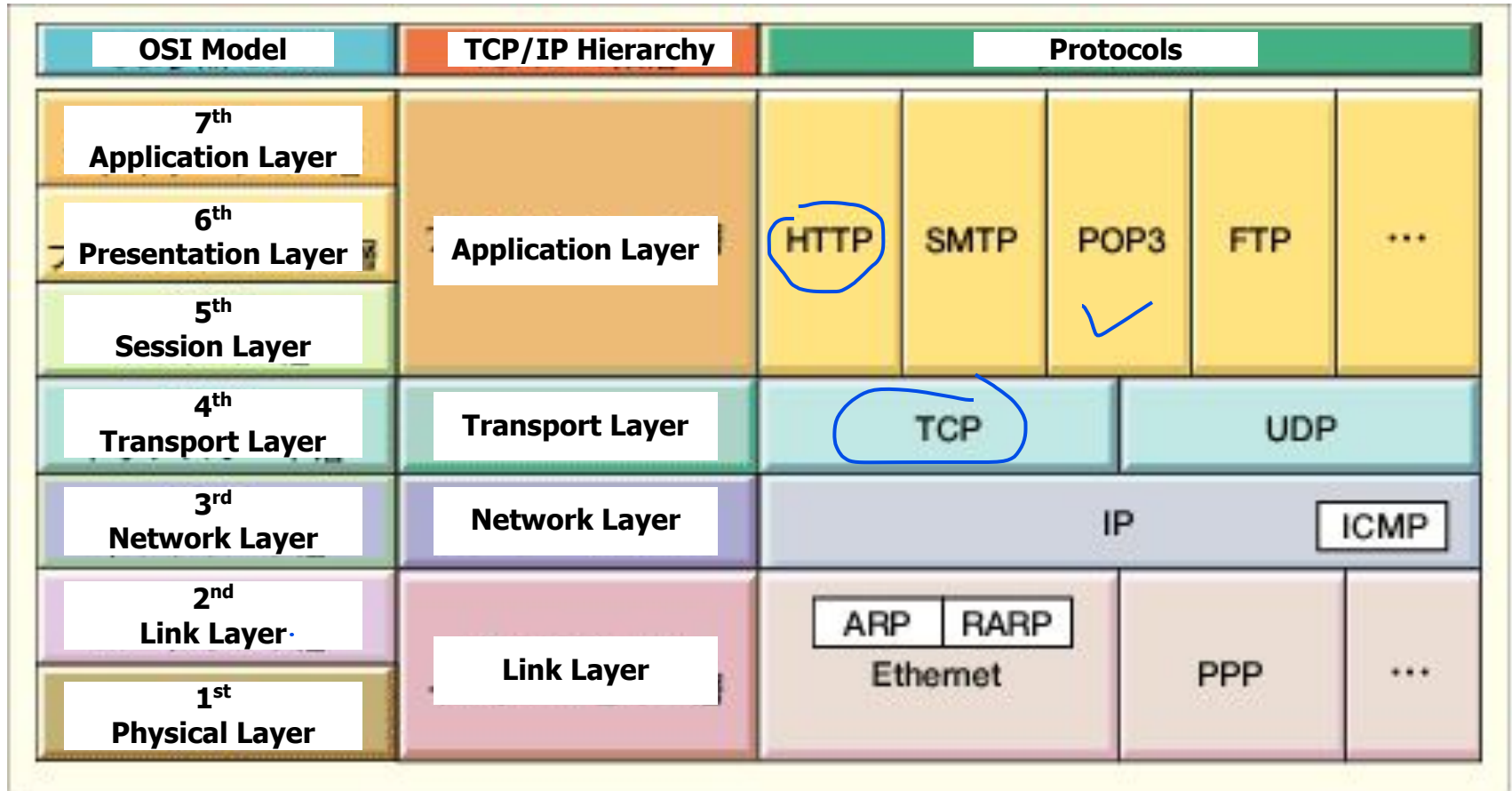
- Seven network “layers”
 - Layer 1 : Physical – cables
 - Layer 2 : Data Link – ethernet
 - Layer 3 : Network – IP
 - Layer 4 : Transport – TCP/UDP
 - Layer 5 : Session
 - Layer 6 : Presentation
 - Layer 7 : Application

TCP/IP Network Model



- Different view – 4 layers
 - Layer 1 : Link
 - Layer 2 : Network
 - Layer 3 : Transport
 - Layer 4 : Application

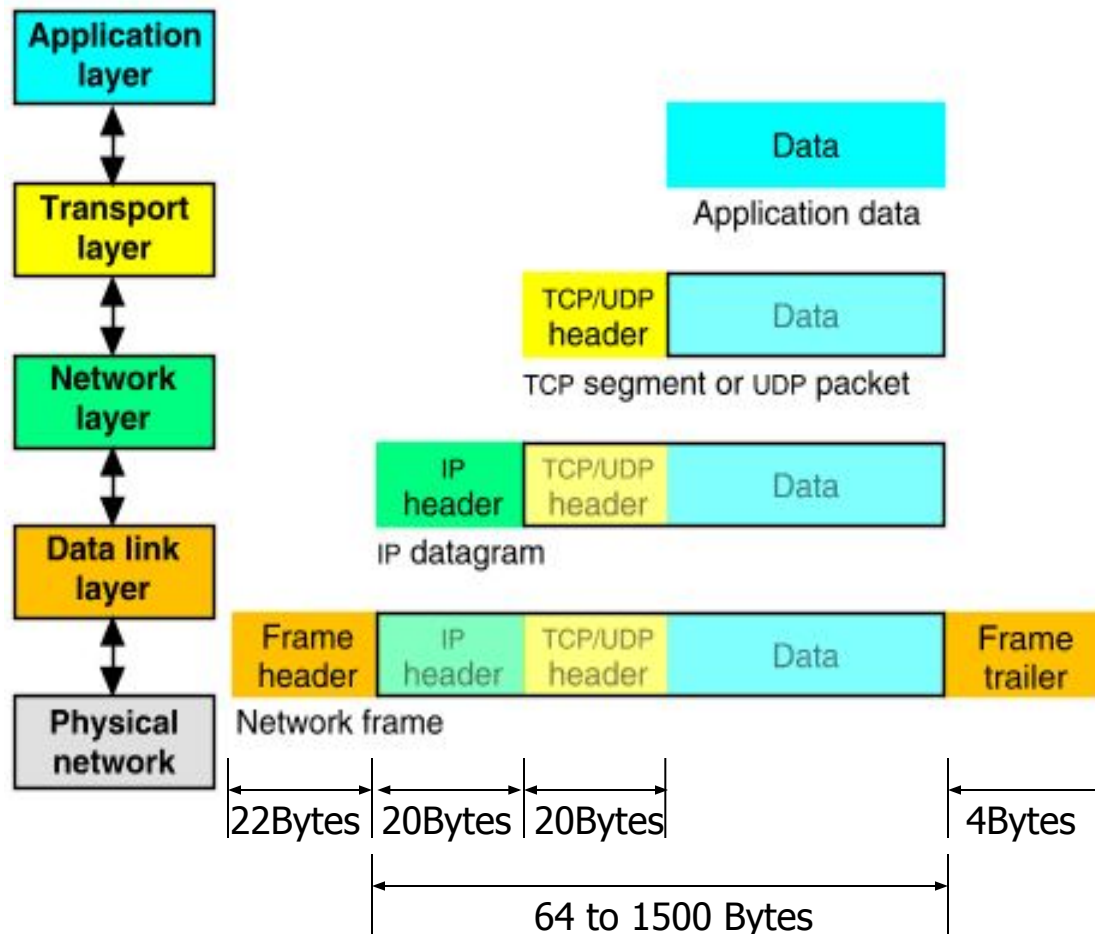
OSI: Open Systems Interconnect



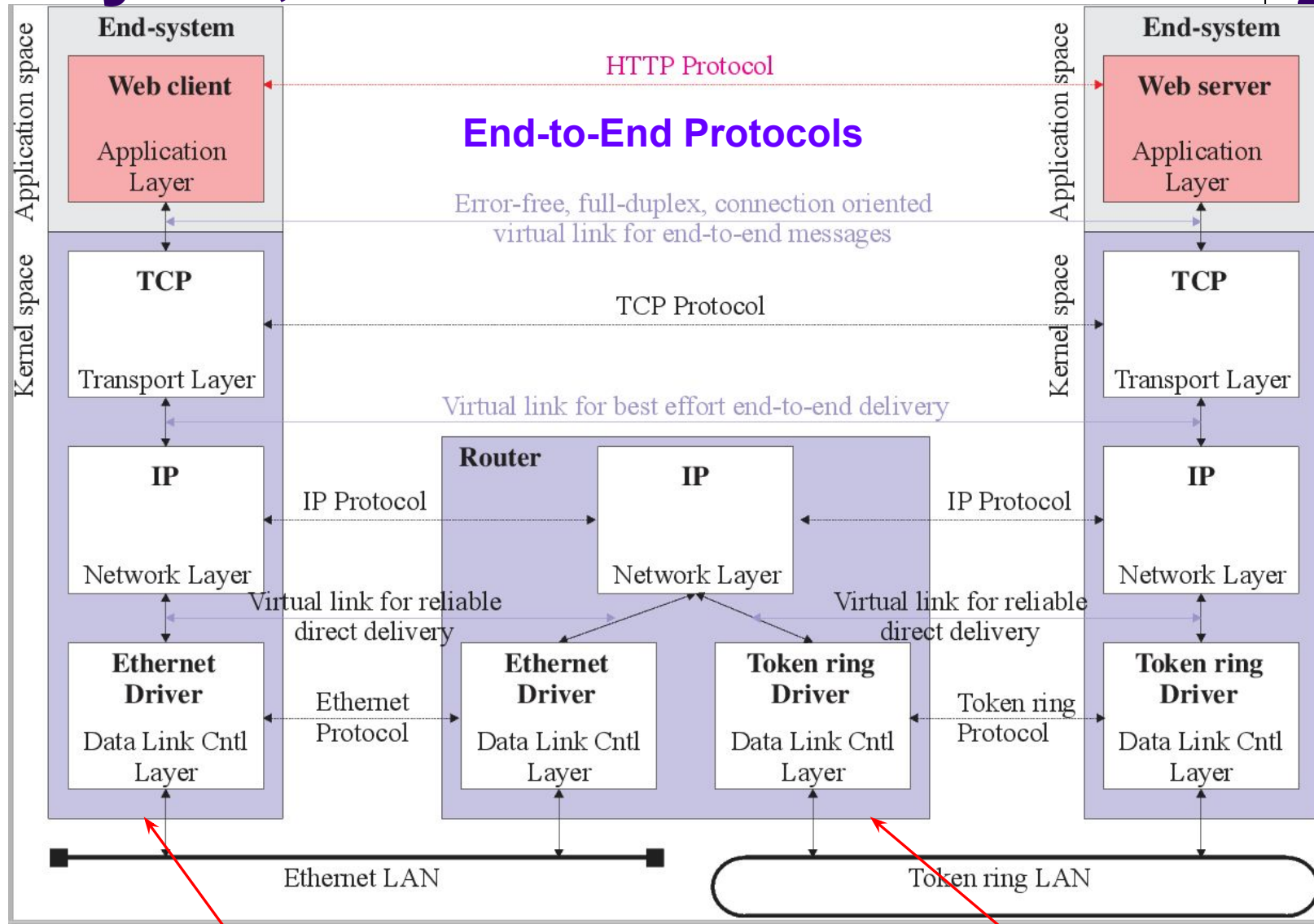
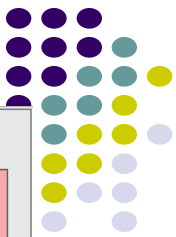
- Link Layer : includes device driver and network interface card
- Network Layer : handles the movement of packets, i.e. Routing
- Transport Layer : provides a reliable flow of data between two hosts
- Application Layer : handles the details of the particular application

Packet Encapsulation

- The data is sent down the protocol stack
- Each layer adds to the data by prepending headers



Layers, Services & Protocols



Communicating End Systems

One or More Network Nodes

Why Layering?



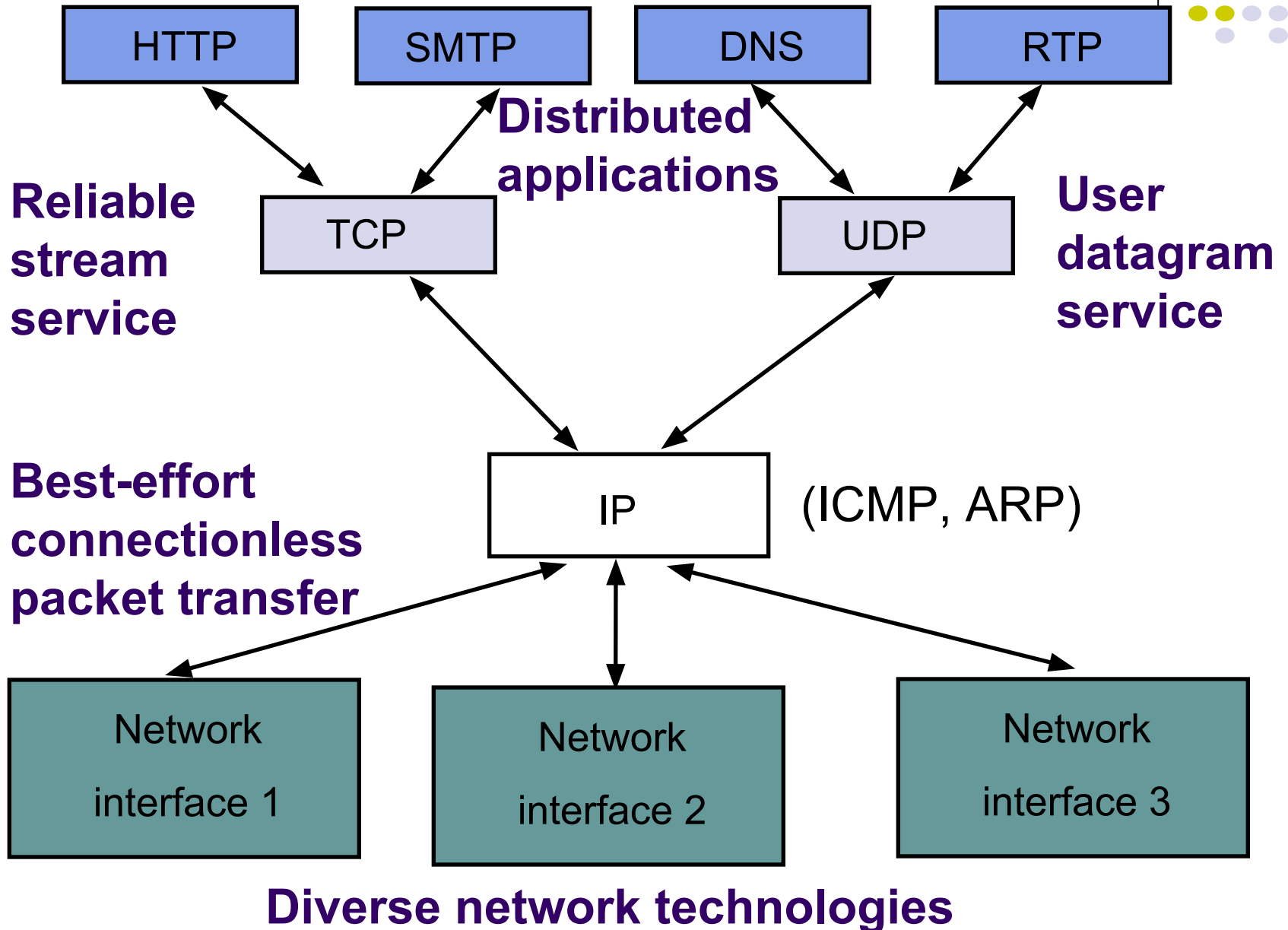
- Layering simplifies design, implementation, and testing by partitioning overall communications process into parts
- Protocol in each layer can be designed separately from those in other layers
- Protocol makes “calls” for services from layer below
- Layering provides flexibility for modifying and evolving protocols and services without having to change layers below
- Monolithic non-layered architectures are costly, inflexible, and soon obsolete

Protocols

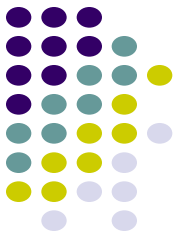


- Communications activity between computers requires very specific unambiguous rules
- A *protocol* is a set of rules that governs how two or more communicating entities in a layer are to interact
- *Messages* that can be sent and received
- *Actions* that are to be taken when a certain event occurs, e.g. sending or receiving messages, expiry of timers
- **The purpose of a protocol is to provide a service to the layer above**

Protocols of the Internet

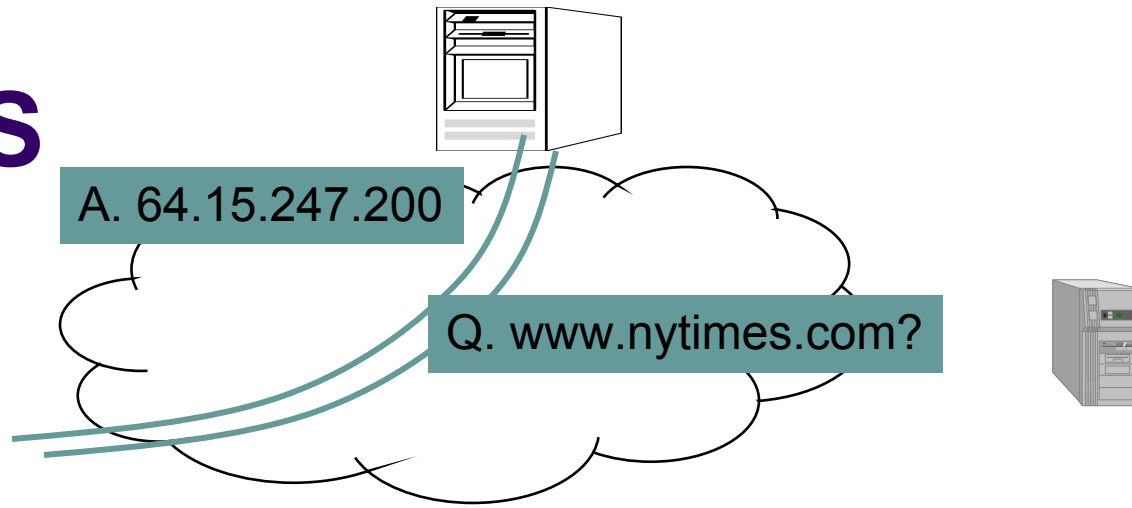


Protocols in Action: Web Browsing Application



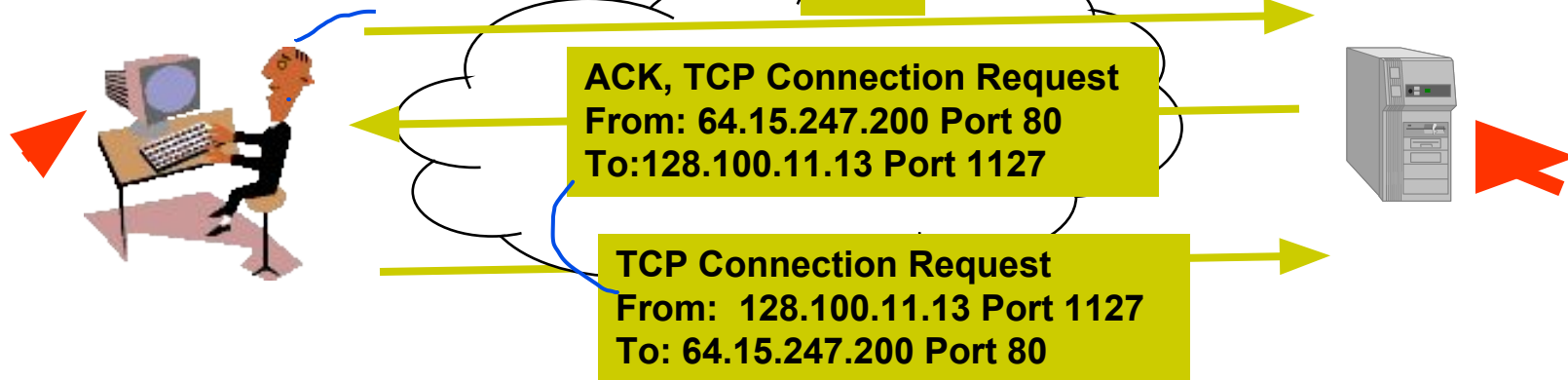
- World Wide Web allows users to access resources (i.e. documents) located in computers connected to the Internet
- Documents are prepared using HyperText Markup Language (HTML)
- A browser application program is used to access the web
- The browser displays HTML documents that include *links* to other documents
- Each link references a *Uniform Resource Locator* (URL) that gives the name of the machine and the location of the given document
- Let's see what happens when a user clicks on a link

1. DNS



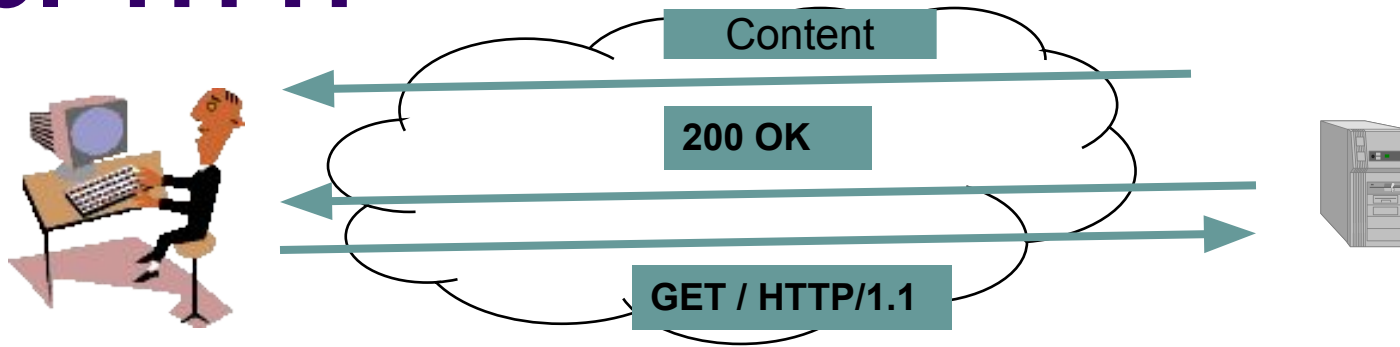
- User clicks on <http://www.nytimes.com/>
- URL contains Internet name of machine (www.nytimes.com), but not Internet address
- Internet needs Internet address to send information to a machine
- Browser software uses **Domain Name System** (DNS) protocol to send query for Internet address
- DNS system responds with Internet address

2. TCP



- Browser software uses HyperText Transfer Protocol (HTTP) to send request for document
- HTTP server waits for requests by listening to a well-known port number (80 for HTTP) (more later)
- HTTP client sends request messages through an “ephemeral port number,” e.g. 1127
- HTTP needs a **Transmission Control Protocol** (TCP) connection between the HTTP client and the HTTP server to transfer messages reliably

3. HTTP

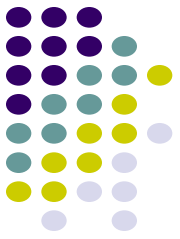


- HTTP client sends its request message: “GET ...”
- HTTP server sends a status response: “200 OK”
- HTTP server sends requested file
- Browser displays document
- Clicking a link sets off a chain of events across the Internet!
- Let’s see how protocols & layers come into play...

Layers



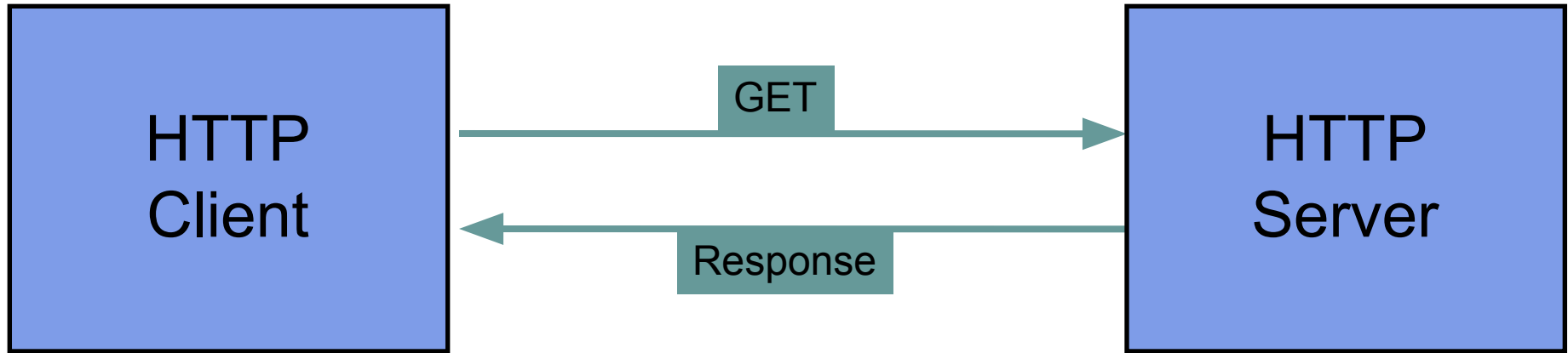
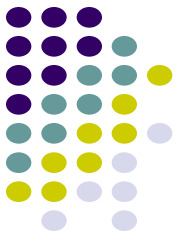
- A set of related communication functions that can be managed and grouped together
- Application Layer: communications functions that are used by application programs
 - HTTP, DNS, SMTP (email)
- Transport Layer: end-to-end communications between two processes in two machines
 - TCP, User Datagram Protocol (UDP)
- Network Layer: node-to-node communications between two machines
 - Internet Protocol (IP)



Example: HTTP

- Hypertext Transfer Protocol
- HTTP is an application layer protocol
- Retrieves documents on behalf of a browser application program
- HTTP specifies fields in request messages and response messages
 - Request types; Response codes
 - Content type, options, cookies, ...
- HTTP specifies actions to be taken upon receipt of certain messages

HTTP Protocol



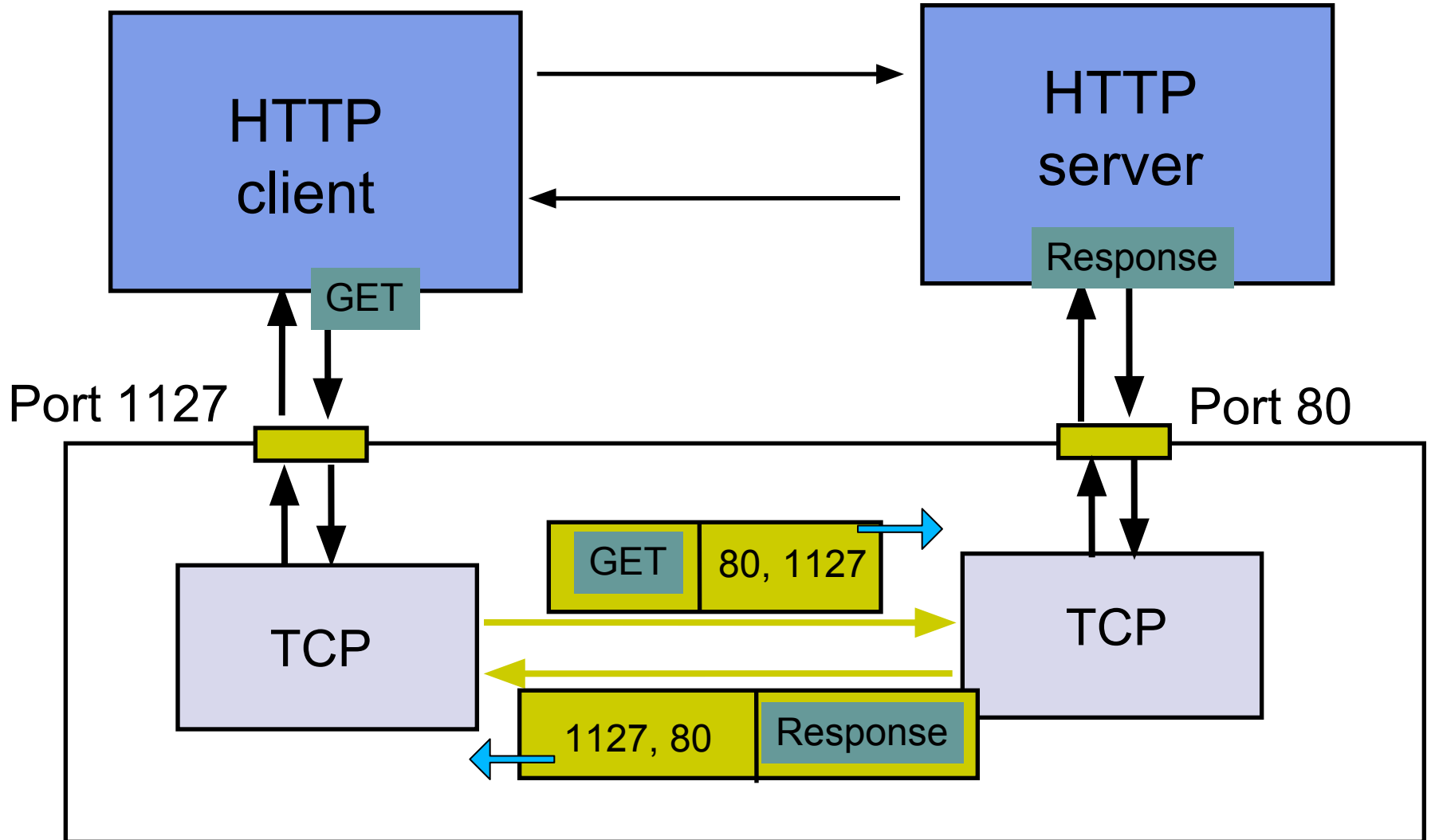
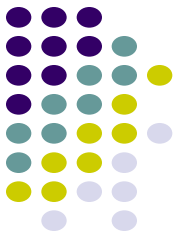
- HTTP assumes messages can be exchanged directly between HTTP client and HTTP server
- In fact, HTTP client and server are processes running in two different machines across the Internet
- HTTP uses the reliable stream transfer service provided by TCP

Example: TCP



- TCP is a transport layer protocol
- Provides *reliable byte stream service* between two processes in two computers across the Internet
- Sequence numbers keep track of the bytes that have been transmitted and received
- Error detection and retransmission used to recover from transmission errors and losses
- TCP is *connection-oriented*: the sender and receiver must first establish an association and set initial sequence numbers before data is transferred
- Connection ID is specified uniquely by
(*send port #, send IP address, receive port #, receiver IP address*)

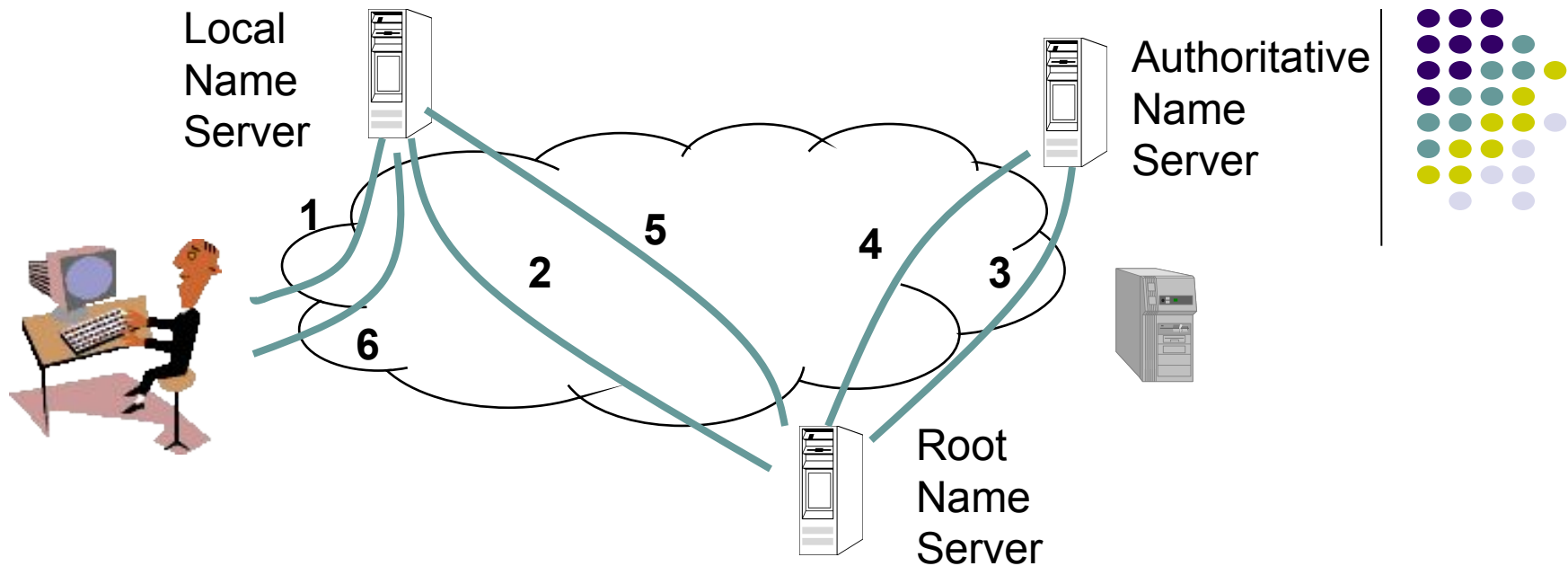
HTTP uses the services of TCP



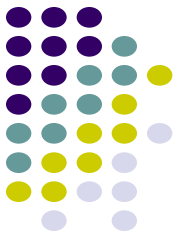
Example: DNS Protocol



- DNS protocol is an application layer protocol
- DNS is a distributed database that resides in multiple machines in the Internet
- DNS protocol allows queries of different types
 - Name-to-address or Address-to-name
 - Mail exchange
- DNS usually involves short messages and so uses service provided by UDP
- Well-known port 53



- **Local Name Server:** resolve frequently-used names
 - University department, ISP
 - Contacts Root Name server if it cannot resolve query
- **Root Name Servers:** 13 globally
 - Resolves query or refers query to Authoritative Name Server
- **Authoritative Name Server:** last resort
 - Every machine must register its address with at least two authoritative name servers



Example: UDP

- UDP (User Datagram Protocol) is a transport layer protocol
- Provides *best-effort datagram service* between two processes in two computers across the Internet
- Port numbers distinguish various processes in the same machine
- UDP is *connectionless*
- Datagram is sent immediately
- Quick, simple, but not reliable

Summary



- Layers: related communications functions
 - Application Layer: HTTP, DNS
 - Transport Layer: TCP, UDP
 - Network Layer: IP
- Services: a protocol provides a communications service to the layer above
 - TCP provides connection-oriented reliable byte transfer service
 - UDP provides best-effort datagram service
- Each layer builds on services of lower layers
 - HTTP builds on top of TCP
 - DNS builds on top of UDP
 - TCP and UDP build on top of IP